

ANDRIEU Emma

M2 SIGMA

2025-2026

Gipsa-lab

11 rue des mathématiques, 38400 Saint-Martin-d'Hères, FRANCE

Universal Machine-Learning Models : applications to Brain-Computer Interfaces

from 01/02/2026 to 30/06/2026

Under the supervision of :

Laboratory supervisor: Marco CONGEDO, marco.congedo@gipsa-lab.grenoble-inp.fr

Phelma Tutor: Ronald PHLYPO, ronald.phlypo@grenoble-inp.fr

Confidentiality : No

Ecole nationale
supérieure de physique,
électronique, matériaux

Phelma

Tél +33 (0)4 56 52 91 00

<http://phelma.grenoble-inp.fr>



This work was supported by the French National Research Agency (ANR) under the "France 2030" program (ANR-15-IDEX-02), within the LabEx PERSYVAL (ANR-11-LABX-0025-01)

Acknowledgements

I would like to express my sincere gratitude to my internship supervisor, Marco Congedo, for his guidance throughout this internship. His advice, availability, and scientific expertise allowed me to develop both technically and methodologically.

I would also like to thank Isabella Costa Maia for her valuable support and availability. Her explanations, advices, and the discussions we shared greatly facilitated the completion of this project.

Finally, I would like to thank my family and friends for their continuous support and encouragement throughout my studies and during the preparation of this thesis.

Abstract

In the context of EEG-based motor imagery brain-computer interfaces (BCIs), Riemannian neural networks operating on Symmetric Positive Definite (SPD) matrices have gained increasing attention. A key component of these architectures is the activation function, which introduces non-linearity while preserving the geometric structure of SPD matrices. However, the standard ReEig activation, may provide a limited non-linear capacity. This study investigates whether alternative non-linear activation functions can improve classification performance within the SPDNet architecture. A comparative analysis is conducted, followed by statistical analysis, between the standard activation against spectral functions and element-wise functions. The experiments are performed on multiple datasets from a public EEG database. Results show that alternative functions such as ExpT and CoshP outperform the conventional activation in term of classification accuracy.

Contents

1	Introduction	4
2	Description of the lab	6
3	Context	8
4	State of the art	9
4.1	SPD matrices	9
4.2	Riemannian geometry	10
4.3	Riemannian methods for EEG classification	12
4.3.1	Standard Baseline : CSP + LDA	12
4.3.2	Riemannian CSP	13
4.3.3	Minimum Distance to Riemannian Mean	13
4.3.4	Tangent space methods	14
4.3.5	Kernel method	14
4.4	SPDNet	14
4.4.1	BiMap Layer	15
4.4.2	ReEig Layer	15
4.4.3	LogEig Layer	16
4.4.4	Backpropagation and Optimization on Riemannian manifold	16
4.5	Non Linear Activation Functions	18
4.5.1	Spectral-based activation	18
4.5.2	Entry-wise activations	18
4.5.3	Log-Euclidean activations	19
5	Model and methods	20
5.1	Data	20
5.2	Pre-processing of the data	20
5.3	Library SPD Learn	21
5.4	Model	21
5.5	Activation Layers	22
5.5.1	PowerEig	23
5.5.2	TanhEig	24
5.5.3	SpAEig	26
5.5.4	Parametric Hyperbolic Cosine - CoshP	28
5.5.5	Polynomial Activation - PolyAct	29
5.5.6	Parametric Exponential Activation - ExpP	29
5.5.7	Truncated Exponential - ExpT	30
5.6	Statistical Tests	30

6	Results	32
6.1	Classification performance	32
6.2	Statistical analysis	34
6.2.1	Omnibus analysis	34
6.2.2	Post-hoc analysis	34
7	Discussion	37
7.1	Summary of results	37
7.2	Interpretation	37
7.3	Limitations	38
7.4	Perspectives	38
8	Conclusion	39
A	Gradient derivation for the TanhEig layer	42
B	Gradient derivation for the SpAEig layer	43

1 Introduction

A Brain-Computer Interface (BCI) is a system that translates brain signals into commands. It allows a person to communicate with or to control an external device without using neuromuscular pathways. BCI operations are based on the interaction of two agents : the user and the machine. The user produces brain signals, where the intent is encoded, and the machine translates these signals into outputs. The interface must provide a continuous feedback to the user on the performed actions.

This technology will be useful for individuals who are suffering from severe motor disabilities such as amyotrophic lateral sclerosis, spinal cord injury or any neuromuscular diseases or injuries. The quality of life of people who are "locked-in" would be significantly improved. A BCI can be used for different applications such as communication, movement control, environmental control, locomotion or rehabilitation. Besides these medical applications, BCIs could also be used in non-clinical fields such as computer gaming [1].

Among the different brain signal acquisition techniques, electroencephalography (EEG) is one of the most widely used because of its non-invasiveness, making it one of the safest BCI recording methods. In particular motor imagery paradigms, where users imagine performing specific movements without actual muscular activity, are commonly used in EEG-based BCIs. The objective is to classify EEG signals associated with different imagined movements such as left hand, right hand, tongue or foot movements. Motor imagery relies on the observation that imagined actions can elicit neural activity patterns that are partially similar to those observed during actual motor execution. This concept is often referred to as the functional equivalence hypothesis, which suggests that motor imagery and motor execution share overlapping neural mechanisms [2] [3]. However, decoding motor imagery EEG signals remains a challenging task. EEG signals exhibit transient modulations of rhythmic brain activity that reflect underlying cognitive processes. These signals are characterized by a low signal-to-noise ratio (SNR), strong variability across subjects and sessions, and low specificity between mental tasks. As a result, EEG-based brain decoding represents a challenging problem in statistical learning, particularly in the context of BCIs [4].

To address these challenges, a variety of machine learning and deep learning approaches have been proposed. Recently, several deep neural network architectures have been developed, allowing the processing of Symmetric Positive Definite (SPD) matrices, which naturally arise in covariance-based representations of EEG [5]. These architectures exploit the Riemannian geometry of SPD matrices and have demonstrated promising performances in EEG signal classification [6].

Nevertheless, some architectural components could be further explored to potentially improve network performance. In particular the activation layer, a fundamental ingredient of neural network, that plays a crucial role by introducing non-linearity in the model.

The objective of this internship is to investigate non-linear activation functions for Riemannian neural networks and to evaluate their impact on EEG motor imagery classification

performance. The proposed activations are compared with the standard activation layer currently used in SPD neural networks.

This report is organized as it follows. Section 2 presents the laboratory in which this internship was carried out. Section 3 introduces the context of EEG-based motor imagery classification. Section 4 reviews the state-of-the-art methods. Section 5 describes the model, data, and methods used for SPD neural networks. Section 6 presents the experimental results, followed by a discussion in Section 7 and a conclusion in Section 8.

2 Description of the lab

The Gipsa-lab [7] (Grenoble Image Parole Signal Automatique) is a research laboratory of Grenoble INP, the university of Grenoble Alpes and CNRS. It conducts research in several thematics such as Automation, Signal, Images, Speech, Cognition, Robotics, and Learning.

The lab is organized into four scientific poles : the Automatic and Diagnostic Pole (PAD), the Data Sciences Pole (PSD), the Speech and Cognition Pole (PPC) and the Geometry, Learning, Information and Algorithms Pole (GAIA). The research support staff are organised into two divisions: the Administrative and Financial Division and the Technical Division. The laboratory is home to a scientific community comprising permanent researchers, engineers, PhD students, as well as interns and postdoctoral researchers.

The Data Science Pole (PSD) at Gipsa-lab brings together researchers from various scientific communities who share common interests in the field of data science, whenever derived from observation or intended for analysis. Its main objective is to strengthen interactions between complementary fields such as signal and image processing, artificial intelligence, machine learning, robotics, biomechanics, cognition and the study of physiological signals. The PSD activities are based on three main areas :

- Modelling and studying the physical and cognitive processes underlying signals and images, as well as the associated complex systems.
- Developing advanced methods for data processing, machine learning and control in robotics and biomechanics.
- Acquiring experimental data, validating the approaches developed, in order to bridge the gap between theoretical aspects and practical applications.

Through its five research teams, the pole addresses major challenges in the fields of environment, health, biomedical, perception and robotics. Those challenges are related to artificial intelligence, data fusion, sensor networks and dynamic systems, autonomous robotics and the modelling of human movement.

All this work forms part of a strongly interdisciplinary approach, ranging from theory to experimentation, with a firm focus on societal issues such as health, the environment, smart mobility and the ecological transition [8].

In the Data Science Pole, the ViBS team conducts research into Vision and Brain Signal processing. The fields of application are electroencephalography (EEG), electrocardiography (ECG), electroencephalocardiography (EEG-ECG) and electromyography (EMG), as well as eye tracking, behavioural studies and video analysis. The team uses multivariate methods such as optimal filtering, independent component analysis and Riemannian geometry [9]. The main research topics of the team are :

1. Brain Computer Interface (BCI)
2. Gaze-EEG co-recording used for multimodal analysis of cognitive functions

3. Multi-stable visual perception by developing methods to understand the dynamics of visual perception through stimulation with ambiguous visual stimuli
4. The development of analytical tools and theoretical advances in the field of Riemannian geometry applied to data processing
5. Transfer learning for BCI by developing EEG-based transfer learning methods using Riemannian geometry
6. Intra-saccadic perception
7. Multimodal non-invasive foetal monitoring, involving multimodal monitoring (via ECG, PCG, CTG) of the foetus
8. Experimental design, optimising the number and placement of sensors to extract information buried in noise
9. Multimodal observation factorisation, applied to ECG, PCG and EMG

Therefore, the GIPSA-lab and the ViBS team offered a particularly relevant scientific environment for conducting research on deep learning approaches for Brain-Computer Interfaces.

3 Context

Motor Imagery EEG-based BCIs aim to decode user’s intention directly from brain signals and convert them into commands without passing through the neuromuscular pathway. Identifying the command the user intends to issue relies on a classification algorithm designed to determine the category of the data provided. Classification algorithms are used to identify patterns of brain activity and are the most popular approach. The recognition of a pattern depends on both the features and the classifier algorithm. However, both components present some limitations.

EEG signals have a low signal-to-noise ratio, resulting in noise and outliers in the features. Besides, EEG features are non-stationary because of the variance of EEG signals over time and over sessions.

A typical classifier used in BCIs is the Linear Discriminant Analysis (LDA), which uses hyperplanes to separate data representing the different classes. One of the drawbacks of this classifier is its linearity, resulting in poor results on complex non-linear patterns in EEG data. Another classifier commonly used in BCIs is the Support Vector Machine (SVM), that seeks hyperplanes maximizing margins. Despite the low speed of execution, SVM has good generalization properties [10].

Apart from the classical machine learning classification algorithms, methods based on Riemannian geometry are also commonly used in the field of BCIs. Riemannian classifiers either work directly on the manifold of Symmetric Positive Definite (SPD) matrices or rely on the projection of data into the tangent space. These geometric methods, such as Riemannian Minimum Distance to Mean (RMDM) or tangent space mapping combined with a linear classifier, exploit the geometric structure of SPD matrices [5]. While they have proven very accurate [11], they may not be able to capture the complex non-linear patterns in EEG signals [12].

To address this issue, deep learning models adapted to SPD manifold have been introduced, starting from the SPDNet [13]. These architectures aim at combining the advantages of deep neural networks to capture the complexity of patterns, with the geometric constraint of SPD matrices. However, some components of this model remain suboptimal for capturing complex EEG patterns. The main activation mechanism used to introduce non-linearity in SPD neural network is the ReEig layer, thresholding the eigenvalues of the SPD matrix to ensure positivity and preserve the SPD structure. However, this activation function may provide a limited non-linearity capacity, and thus restrain the expressive power of the network [14].

In this study, we focus on activation functions in order to introduce other kind of non-linearities in SPD neural networks so as to improve their ability to learn discriminative features. The goal is to improve classification performance for motor imagery EEG decoding, while preserving the geometry of SPD matrices. Motivated by these limitations, we first present the principal Riemannian approaches developed for EEG classification before discussing methods based on deep learning architectures designed to operate directly on SPD manifold.

4 State of the art

This section reviews the main concepts and methods underlying EEG classification using Riemannian geometry. We first introduce the representation of EEG trials through covariance matrices, before presenting the geometry of SPD matrices. Next, various Riemannian methods developed for EEG classification are detailed. Finally, the SPDNet network is introduced, with a presentation of its different layers, backpropagation methods and optimisation techniques. To conclude, alternative activation functions to the ReEig layer are presented.

4.1 SPD matrices

The EEG signal is decomposed into trials, corresponding to temporal windows of the data and denoted by E_z with $z \in \{1, \dots, Z\}$ the index of Z classes. Trials E_z are EEG matrices of shape $N \times T$ with N the number of electrodes and T the number of samples. In order to capture all relevant information about the signal, the covariance matrix Σ of a trial is defined by :

$$\Sigma_z = \frac{1}{T-1} (E_z E_z^T),$$

[5] assuming the time-series are centered, which is always the case on band-pass filtered signal. The sample covariance matrix Σ contains all the spatial information, particularly its second order statistics. The diagonal elements of the matrix hold the variance at each electrode and the off-diagonal elements hold the covariance between pairs of electrodes. Covariance matrices Σ_z belong to the set of $n \times n$ positive definite matrices P_n with $S^{++}(n)$ the space of all $n \times n$ symmetric matrices [15].

Figure 1 illustrates the geometric structure of the SPD manifold. Matrices on the boundary of the cone are positive semi-definite, whereas strictly positive definite matrices Σ_1 and Σ_2 lie in its interior.

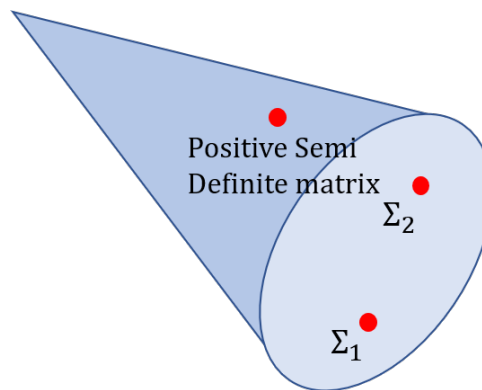


Figure 1: Cone of SPD matrices.

Those covariance matrices are sufficient for the classification of motor imagery tasks because the scalp spatial pattern is different for each class. As explained in [16], motor movements result in simultaneously event-related de-synchronization (ERD) and event-related synchronization (ERS) on different areas of the scalp. ERD corresponds to a decrease in the power (and

consequently the variance) of sensorimotor rhythms, whereas ERS corresponds to an increase in power and variance. Cortical areas involved in the imagined movement exhibit ERD, while surrounding or task-irrelevant areas may exhibit ERS. This phenomenon can be explained by the somatotopic organization of the sensorimotor cortex, commonly represented by the sensory-motor homunculus in Figure 2. Different body parts are associated with distinct cortical regions. Consequently, a hand movement will produce an ERD in the hand area and an ERS in the foot area. A foot movement will result in an opposite pattern, with an ERD in the foot area and an ERS in the hand area. The simultaneous occurrence of ERD and ERS depending on the task performed allows the sample covariance matrix to capture all relevant information for the classification of motor imagery tasks.

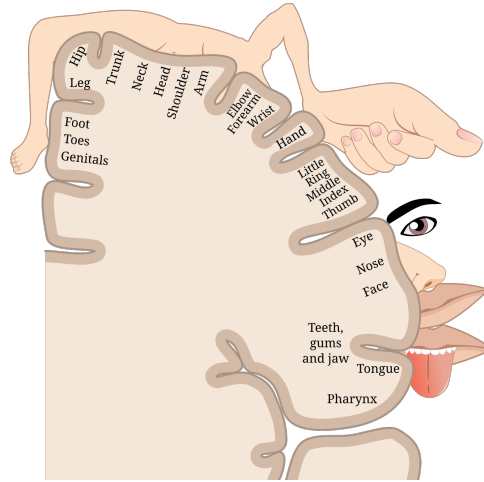


Figure 2: homunculus sensory motor.

Although covariance matrices provide an informative representation of EEG trials, they do not form an Euclidean space because of their belonging to the set of SPD matrices. Their manipulation with Euclidean operations may lead to a geometrical distortion and may not preserve the intrinsic structure of the data.

4.2 Riemannian geometry

To overcome the limitations of Euclidean metrics, SPD matrices are studied within the framework of Riemannian geometry, where the space is modelled as a differentiable manifold with a suitable metric. In the following sections, we define the Riemannian Geodesic distance, the Riemannian mean, as well as the exponential and logarithmic maps [5] [15].

Riemannian Geodesic distance

The geodesic distance is the minimal length of all the curves connecting two points P_1 and P_2 on the manifold, as illustrated on Figure 3.

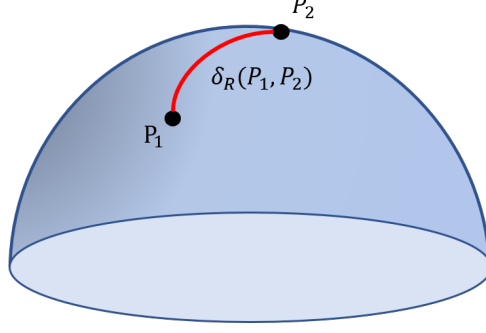


Figure 3: Geodesic distance between two SPD matrices.

The computation of the geodesic distance between two SPD matrices P_1 and P_2 in the set $S^{++}(n)$ is given by the Riemannian distance, defined as

$$\delta_R(P_1, P_2) = \|\log(P_1^{-1/2} P_2 P_1^{-1/2})\|_F = \left[\sum_{i=1}^n \log^2 \lambda_i \right]^{1/2},$$

where $P_1^{-1/2}$ is the inverse of the principal square root of P_1 and $\lambda_i, i = 1 \dots n$, are the eigenvalues of $P_1^{-1} P_2$.

The Riemannian distance has all the properties of a distance function. Furthermore, given an invertible matrix $X \in \mathbb{R}^{n \times n}$:

- The Riemannian distance is invariant under any congruence :

$$\delta_R(X^T P_1 X, X^T P_2 X) = \delta_R(P_1, P_2).$$

- The Riemannian distance is invariant under inversion :

$$\delta_R(P_1^{-1}, P_2^{-1}) = \delta_R(P_1, P_2).$$

Riemannian mean

To define the Riemannian mean, we use Fréchet's variational approach. A point X is called the mean of the set of points $\{P_1, \dots, P_k\}$ if there exists a unique point X that minimizes the dispersion $\frac{1}{K} \sum_{k=1}^K \delta^2(P_k, X)$. Thus, the Riemannian mean is defined as

$$M_R = \arg \min_X \frac{1}{K} \sum_{k=1}^K \delta_R^2(P_k, X).$$

Using the Riemannian geodesic distance, the minimizer exists and is unique. M_R is called the geometric mean, as it reduces to the scalar geometric mean for $n = 1$.

Exponential and logarithmic mapping

The space of SPD matrices is modelled as a differentiable manifold. Around each point P , the manifold can be locally approximated by a Euclidean vector space. This space is called the tangent space $T_P M$ and is composed of all tangent vectors attached to P .

A tangent vector S_i can be interpreted as the derivative at $t = 0$ of a geodesic curve $\Gamma_i(t)$ starting from P and reaching the point P_i on the manifold. The exponential map at the reference point P maps a tangent vector S_i to a point P_i on the manifold and is given by :

$$\text{Exp}_P(S_i) = P_i = P^{1/2} \exp(P^{-1/2} S_i P^{-1/2}) P^{1/2}.$$

Conversely, the logarithmic mapping projects a point P_i onto the tangent space :

$$\text{Log}_P(P_i) = S_i = P^{1/2} \log(P^{-1/2} P_i P^{-1/2}) P^{1/2}.$$

Figure 4 illustrates the map of a SPD matrix from the tangent space to the manifold, and inversely.

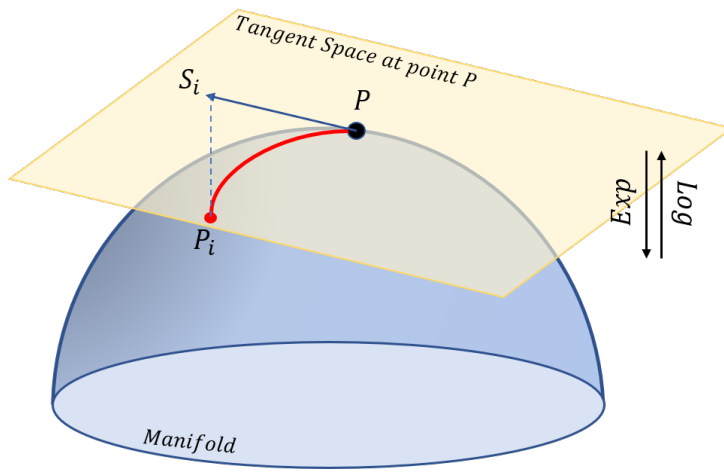


Figure 4: Exponential and logarithmic maps on the SPD manifold.

These tools allow the classification of covariance matrices, extracted from EEG data, directly on the SPD manifold or after projection into a tangent space. Several classification approaches that exploit the Riemannian structure of the data are presented in the following section.

4.3 Riemannian methods for EEG classification

The goal of BCIs is to determine the mental task, corresponding to the class, associated to an observation. For instance, the mental task would be the imagery movement of the right/left hand, the feet, or the tongue. This classification problem is usually solved with supervised learning. In the following subsections, different methods used for this classification problem are detailed.

4.3.1 Standard Baseline : CSP + LDA

The baseline method to classify imagery movements is the use of the spatial filter Common Spatial Pattern (CSP) [17] to extract discriminatory information from EEG trials. A weighting of the electrodes is performed according to their importance in the discriminatory task. The CSP method seeks linear combinations of electrodes that maximize the variance of one class while minimizing it for the other class. For an EEG trial, data are represented as a matrix

$E \in \mathbb{R}^{N \times T}$ where N is the number of electrodes and T the number of samples. From this matrix, a normalized covariance matrix C is calculated as in 4.1. The covariance matrices are then averaged separately for each class, yielding two mean covariances $\bar{C}_1$ et $\bar{C}_2$. A composite matrix is then defined by $C_c = \bar{C}_1 + \bar{C}_2$. The eigenvectors of $C_c^{-1}\bar{C}_1$ correspond to the CSP spatial filters. The filters associated with the largest eigenvalues define optimal spatial projections maximizing the variance ratio:

$$\max_w \frac{w^T C_1 w}{w^T C_2 w},$$

whereas the filters associated with the smallest eigenvalues maximize the variance of class 2 while minimizing that of class 1 :

$$\max_w \frac{w^T C_2 w}{w^T C_1 w}.$$

The EEG data are then projected using the spatial filters. For each component, the variance is calculated and then logarithmically transformed to construct the features used for classification. These features are ultimately used as input to a linear classifier, typically LDA.

4.3.2 Riemannian CSP

In [18], a link between CSP and Riemannian geometry is elucidated. Usually, the class-related mean covariance matrices $\bar{C}_1$ et $\bar{C}_2$ are obtained using arithmetic mean. However, SPD matrices lie on the Riemannian manifold. The arithmetic mean of $\bar{C}_1$ and $\bar{C}_2$ is thus replaced by the Riemannian mean M_R to obtain the class-related mean covariance matrices.

Spatial filters are then obtained as the eigenvectors of $M_R^{-1}\bar{C}_1$. The Riemannian approach shows that each filter directly contributes to the Riemannian distance between matrices of the two classes, such as

$$\delta_R(C_1, C_2) = \sqrt{\sum_{j=1}^N \log^2\left(\frac{\lambda_j}{1 - \lambda_j}\right)},$$

where λ_i , $i = 1, \dots, n$ are the eigenvalues of $M_R^{-1}\bar{C}_1$. This formulation therefore allows the selection of spatial filters based on their actual contribution to the geometric distance between the classes. Once the spatial filters have been estimated, the remaining pipeline is identical to standard CSP.

4.3.3 Minimum Distance to Riemannian Mean

To perform classification in the Riemannian manifold, a simple supervised algorithm is the Minimum Distance to Riemannian Mean (MDRM) [15] [19]. For each class k , the Riemannian mean of the training covariance matrices $M_{train}^{(k)}$ is first computed. Given an unknown sample covariance matrix P , the Riemannian distance between P and each class Riemannian mean $M_{train}^{(k)}$ is then computed. The sample is assigned to the class k whose Riemannian mean is the closest to P in terms of Riemannian distance, such as

$$\hat{k} = \arg \min_k \delta_R(P, M_{train}^{(k)}).$$

MDRM performs classification directly on the SPD manifold without requiring any projection

into a Euclidean space.

4.3.4 Tangent space methods

Several approaches make use of the Riemannian tangent space in order to apply classical Euclidean methods to classify SPD matrices. These methods preserve the Riemannian geometry of SPD matrices while enabling the use of conventional machine learning algorithms.

The main idea of Tangent Space Linear Discriminant Analysis [15] is to project SPD matrices onto a tangent space at the geometric mean of the whole set of trials. This reference point is the mean of all samples. Each covariance matrix P_i is mapped into this tangent space at point M_R to obtain a vector s_i of dimension $m = n(n+1)/2$, such as

$$s_i = \text{upper}(M_R^{-\frac{1}{2}} \text{Log}_{M_R}(P_i) M_R^{-\frac{1}{2}}),$$

where $\text{upper}()$ denotes the vectorization of the upper triangular part of the symmetric matrix. A Principal Component Analysis is then applied to decorrelate all variables and reduce the dimensionality of the tangent space representation. Then, a one-way ANOVA is applied to each variable to assess their ability to discriminate between classes. The weighted False Discovery Rate (FDR) is applied to select only the relevant variables. Finally, a classical Euclidean algorithm, such as LDA is applied to classify on the remaining variables.

4.3.5 Kernel method

Another approach consists in using a Riemannian kernel to classify SPD matrices with a Support Vector Machine (SVM) [20]. In this method, covariance matrices are first mapped into the tangent space via the logarithmic map, associated with a reference matrix, typically the Riemannian mean of the dataset. The Riemannian kernel then corresponds to a scalar product calculated in the tangent space, allowing the similarity between two covariance matrices to be assessed while respecting the geometric structure of the SPD manifold. The Riemannian based-kernel is expressed as :

$$k_R(\text{vect}(P_i), \text{vect}(P_j); M_R) = \langle \Theta(P_i), \Theta(P_j) \rangle_{M_R}$$

where $\text{vect}()$ denoted the vectorized tangent space representation, P_i and P_j two covariance matrices, M_R the geometric mean, and $\Theta(P)$ the mapping function on the covariance matrix. This kernel is then used by an SVM classifier to construct a decision boundary that maximizes the margin between classes.

4.4 SPDNet

The success of deep learning in Euclidean domains has led to the emergence of several architectures that enable the learning of discriminative representations directly on SPD manifolds. Among those Riemannian deep learning methods, SPDNet constitutes one of the pioneering

approaches. Huang and Van Gool [13] have introduced a novel network architecture called the SPD matrix network, known under the acronym SPDNet. The network takes SPD matrices as input, and is specifically designed to learn discriminative representations from SPD matrices while preserving their geometric structure.

As illustrated in Figure 5, the architecture is composed of a sequence of manifold-aware layers. Starting from an input SPD matrix, the data are successively processed by a BiMap layer for dimensionality reduction and feature transformation, followed by a rectification layer ReEig that introduces non linearity. The resulting SPD representation is then mapped onto the tangent space through the LogEig layer. Finally, the matrix is vectorized and fed into a softmax classifier for the final prediction. The role of each layer is detailed in the following subsections.

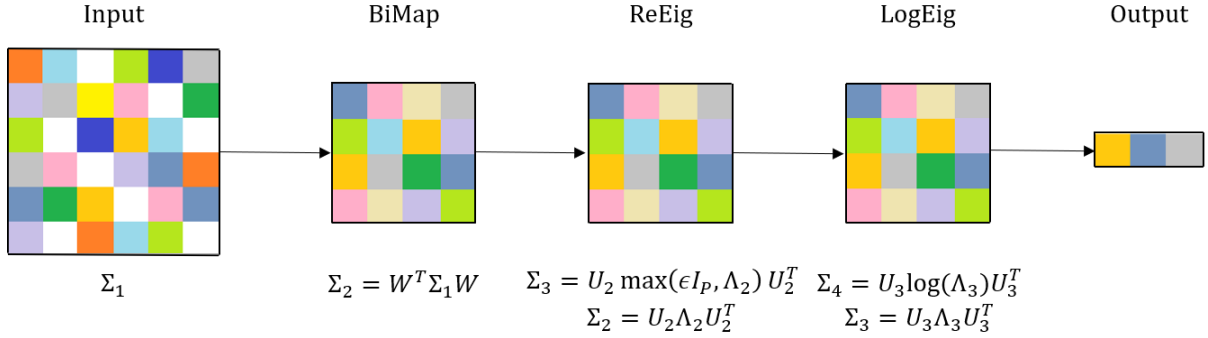


Figure 5: SPDNet model.

4.4.1 BiMap Layer

In order to obtain more compact and discriminative SPD matrices, the BiMap layer performs bilinear mapping on an input SPD matrix $\Sigma_1 \in S_n^{++}$ to produce an output SPD matrix $\Sigma_2 \in S_p^{++}$, with $p < n$. Some trainable weights are applied to the matrix Σ_1 and are learned during training. The transformation is given by :

$$\Sigma_2 = W^T \Sigma_1 W,$$

where $W \in \mathbb{R}^{q \times p}$ is the weights matrix. To ensure that the output matrix Σ_2 remains SPD, we request the matrix W satisfy the orthogonality constraint $W^T W = I_p$.

4.4.2 ReEig Layer

To introduce non-linearity to the neural network while preserving the SPD structure, the ReEig layer is inspired by the ReLU activation used in Euclidean neural networks and rectify SPD matrices by tuning up their small positive eigenvalues. The activation function transforms an input SPD matrix $\Sigma_2 \in S_p^{++}$, with eigenvalue decomposition $\Sigma_2 = U_2 \Lambda_2 U_2^T$, into an output matrix $\Sigma_3 \in S_p^{++}$, such as :

$$\Sigma_3 = U_2 \max(\epsilon I_p, \Lambda_2) U_2^T,$$

where $\epsilon > 0$ is a rectification threshold and I_p an identity matrix of dimension $p \times p$. Small eigenvalues are therefore clipped to a fixed threshold, ensuring numerical stability and preserving positive definiteness.

4.4.3 LogEig Layer

The LogEig layer, used as the final layer, connects the SPD manifold structure to an Euclidean space by projecting an SPD matrix Σ onto the tangent space at the identity matrix I_p . The matrix logarithm is applied to the matrix Σ_3 , followed by vectorisation of the upper triangular coefficients : $vec(\Sigma_4)$ with $\Sigma_4 = U_3 \log(\Lambda_3) U_3^T$. The resulting representation can then be processed by standard neural network layers such as fully connected and softmax layers.

4.4.4 Backpropagation and Optimization on Riemannian manifold

Training SPD neural networks requires computing gradients of matrix-valued functions while respecting the geometry of SPD manifold, and updating parameters using Riemannian optimization techniques. In conventional deep learning frameworks, automatic differentiation tools such as Pytorch’s autograd function, compute the gradients required for backpropagation automatically from the computational graph. In constrast, the SPD Learn library relies on manually derived gradient expressions and custom backward implementations for each layer.

Matrix backpropagation

Several approaches exist to calculate the gradient of matrix functions, and particularly for SPD matrices. The main application of this gradient computation is for backpropagation in matrix-based neural networks. Formulas have been developed by Bhatia, Daleckiĭ and Kreĭn, as well as Ionescu. In [21], a comparison and clarification are made between the Daleckiĭ-Kreĭn/Bhatia and Ionescu approach. Let’s denote X a diagonalizable matrix with the eigenvalue decomposition $X = U \Lambda U^T$ where Λ is a diagonal matrix containing the eigenvalues and U is the matrix of eigenvectors. Besides, let ϕ denote the scalar loss function and Y a matrix-valued operation in the network, such as $Y = f(X) = U f(\Lambda) U^T$. The matrix gradient defined by Daleckiĭ-Kreĭn/Bhatia is given by

$$\frac{\partial \phi}{\partial X} = U \left[K(X) \odot U^T \frac{\partial \phi}{\partial Y} U \right] U^T,$$

with $\odot$ the Hadamard product and $K(X)$ the symmetric Loëwner matrix with elements

$$[K(X)]_{ij} = \begin{cases} \frac{f(\lambda_i) - f(\lambda_j)}{\lambda_i - \lambda_j}, & i \neq j \\ f'(\lambda_i), & i = j \end{cases}.$$

An alternative to Daleckiĭ-Kreĭn/Bhatia formula based on the ideas of Ionescu is given in [21] as follows

$$\frac{\partial \phi}{\partial X} = U [Q^T \odot 2 \text{skew}(C f(\Lambda)) + I_N \odot f'(\Lambda) C] U^T,$$

where $C = U^T (\frac{\partial \phi}{\partial Y}) U$, I_N the identity matrix of size N and Q is the antisymmetric matrix with elements

$$[Q_{ij}] = \begin{cases} \frac{1}{\lambda_i - \lambda_j}, & i \neq j \\ 0, & otherwise. \end{cases}$$

Even though the two methods are equivalent, Daleckiĭ-Kreĭn/Bhatia formula is used in many network implementation because of its stability and computational speed [21]. These gradient

expressions are then used in Riemannian optimization schemes to update parameters while preserving the geometric constraints of SPD manifolds. Once the gradients have been computed, an optimization step is required to update the network parameters. However, when parameters belong to the Riemannian manifold, classical Euclidean procedures no longer preserve the manifold constraints. This motivates the use of Riemannian optimization methods.

Optimization

After the computation of gradients through backpropagation matrix, the network parameters must be updated in order to minimize the loss function. In the Euclidean configuration, stochastic gradient descent (SGD) is commonly used. Let $\phi(w)$ a cost function where $w \in \mathbb{R}^n$ is a parameter to optimize. The SGD update is

$$w_{t+1} = w_t - \gamma_t H(z_t, w_t),$$

where $H(z_t, w_t)$ is the gradient of the loss computed from a training sample z_t , and γ_t is the learning rate. However, this update is no longer valid when the parameter belongs to the Riemannian manifold. To address this problem, Bonnabel [22] proposed a stochastic gradient algorithm on Riemannian manifold (RSGD). The update is performed using the exponential map

$$w_{t+1} = \exp_{w_t}(-\gamma_t H_R(z_t, w_t)),$$

where $H_R(z_t, w_t)$ can be viewed as the Riemannian gradient of the loss. The parameter moves in the direction of the geodesic induced by the exponential map.

In practice, the exponential map is often expensive to compute. Therefore, it is replaced by a retraction R_{w_t} , a first-order approximation of the exponential. The alternative update is thus

$$w_{t+1} = R_{w_t}(-\gamma_t H(z_t, w_t)),$$

Another approach used to update the network parameters on the Riemannian manifold is the trivialization, introduced by Lezcano-Casado [23]. This technique transforms a constrained optimization problem on a manifold into an unconstrained optimization problem in an Euclidean space, through a suitable parametrization. A trivialization is defined as a surjective map $\Theta : \mathbb{R}^n \rightarrow \mathcal{M}$ where $\mathcal{M}$ is the manifold of interest. The optimization problem is thus given by

$$\min_{y \in \mathbb{R}^n} f(\Theta(y)),$$

which is carried out in the Euclidean space while the mapping $\Theta(y)$ ensures that the parameters remain on the manifold.

The Riemannian trivialization is based on the exponential map $\text{Exp}_p : T_p \mathcal{M} \rightarrow \mathcal{M}$, mapping vectors from the tangent space $T_p \mathcal{M}$ onto the manifold. Optimization can be performed on tangent vectors using standard Euclidean optimization algorithms such as SGD or Adam [24]. All these approaches aim at performing optimization under geometric constraints, either by

modifying the update rule (RSGD) or by reparameterizing the manifold (trivialization).

4.5 Non Linear Activation Functions

While SPDNet introduces the ReEig layer as an activation function, alternative functions have been investigated. These approaches aim at improving the expressiveness of the network while preserving the Riemannian structure of the data. The main activation functions proposed in the literature are reviewed in this section. Designing activation functions for SPD neural network is a challenging task because the non-linearity must preserve the structure of SPD matrices. Existing non-linear activations for SPD neural networks can be broadly divided into three categories :

1. Spectral activations, which operate on the eigenvalues of SPD matrices.
2. Entry-wise activations, which apply a non-linearity directly to the matrix coefficients while preserving positive definiteness.
3. Log-Euclidean activations, which perform the non-linear transformation in the tangent space before mapping the result back to the SPD manifold.

4.5.1 Spectral-based activation

Spectral activation functions are the most widely used non-linearities in SPD neural networks. The principle consists in applying a non-linear transformation to the eigenvalues of an SPD matrix. Given the eigendecomposition of an SPD matrix $\Sigma = U\Lambda U^T$, where U holds the eigenvectors and Λ is the diagonal matrix of eigenvalues, a spectral activation is defined as

$$f(\Sigma) = Uf(\Lambda)U^T,$$

where $f(\Lambda)$ denotes the function applied to the eigenvalues. The most common spectral activation is the ReEig layer introduced in SPDNet [13]. By operating directly on the spectrum of SPD matrices, spectral activations preserve the Riemannian structure of data while introducing non-linearity into the model. However, they require an eigendecomposition, which is a costly operation.

4.5.2 Entry-wise activations

While the ReEig layer remains the standard baseline in SPD neural networks, Lucas Heck dos Santo et al. [14] proposed alternative activation functions that do not require an eigenvalue decomposition. The main idea is to apply an element-wise (Hadamard) activation function to an SPD matrix, while preserving its structure. Drawing on the theorems of Schoenberg and Schur, they show that any function $f(t)$ whose Maclaurin series coefficients a_k are non-negative, such as

$$f(t) = \sum_{k=0}^{\infty} a_k t^k,$$

preserves the SPD structure when applied entry-wise to an SPD matrix. This enables the introduction of non-linearities without requiring spectral decompositions. Among the functions

investigated by the authors, the matrix hyperbolic sine ($\sinh$) and hyperbolic cosine ($\cosh$) achieved competitive performances compared with to ReEig layer, while reducing the computational cost.

4.5.3 Log-Euclidean activations

Another family of methods relies on Log-Euclidean mappings. The main idea is to project SPD matrices onto the tangent space using the logarithmic map, apply a standard activation function such as ReLU or SiLU in this Euclidean space, and then map the result back onto the manifold via exponential map. Given an SPD matrix Σ , the activation can be expressed as :

$$Y = \exp(f(\log(\Sigma)))$$

where f denotes a standard activation function and the base point is the identity matrix. This approach allows the use of conventional activation functions while preserving the positive definiteness of resulting matrices.

Building on these developments, our study defines and evaluates new several activation functions for SPD neural networks. Their implementation are described in the following section.

5 Model and methods

This section describes the datasets used in this study, the EEG pre-processing pipeline, and the procedure adopted to generate the training, validation, and testing sets. It also introduces SPD Learn, a Pytorch library for geometric deep learning on SPD matrices. Then, the model architecture and the training configuration are presented. Finally, several activation functions are introduced, as well as the statistical test to compare the performance of different networks.

5.1 Data

In order to train, validate, and evaluate the SPDNet model, several datasets from the corpus FII BCI MI were used [25]. This corpus comprises a selection of BCI databases that have been annotated and curated for research purposes. The datasets selected in this study are summarized in Table 1. For each dataset, information is provided regarding the number of subjects, recording sessions, EEG channels, available classes and sampling rate.

Table 1: Details of datasets from the corpus FII BCI MI

Dataset	Subjects	Sessions	EEG sensors	Classes	Sampling rate (Hz)
BNICI2014_001	9	2	22	4	250
BNICI2015_001	12	2	13	2	512
Cho2017	52	1	64	2	512
GrosseWentrup2009	10	1	128	2	500
Lee2019MI	54	2	62	2	1000
Schirrmeister2017	14	1	128	4	500
Weibo2014	10	1	60	7	200
Zhou2016	4	3	14	3	250

The original references corresponding to each dataset are available in the "BCI DB References" section of the corpus documentation [25].

5.2 Pre-processing of the data

To provide covariance matrices as input to the SPDNet model, the raw EEG recordings must first be pre-processed. For this purpose, pre-processing scripts were developed by Marco Congedo and his PhD student Isabella Costa Maia.

Datasets were selected from the corpus according to several criteria such as the classes and the minimum number of trials. Each raw EEG signal is filtered with a pass-band filter between 8 Hz and 32 Hz. This filtering process retains the frequency bands of interest for motor imagery, namely the μ band between 8 and 13 Hz and the β band between 13 and 30 Hz. Automatic rejection of artifacted trials is performed on EEG data by automatic amplitude thresholding. Each dataset provides pre-segmented trials defined by a fixed temporal window aligned to the onset of the motor imagery cue. Depending on the database, the window length corresponds to a fixed number of samples. Each trial is a EEG segment $E \in \mathbb{R}^{N \times T}$ with N the number of electrodes and T the window length. Covariance matrices are computed for each trial via

Sample Covariance Matrix, as defined in section 4.1. This yields symmetric positive definite matrices of size $N \times N$, which constitute the input representation of the network.

In order to mitigate the problem of variability between subjects and sessions, as mentioned in the section 1, each recording session is treated as a separate domain (i.e, subject-session pair). A domain-aware training strategy is adopted, meaning that domain-specific parameters are learnt for each domain. A 5-fold cross validation strategy is applied.

Within each domain, data are split into training, validation and test sets : 80% of the data are first assigned to training and validation, while 20 % are reserved for testing. The training-validation portion is further divided into 70% for training and 30% for validation. A preconditioning pipeline is then learnt exclusively on the training data and applied to all splits [26]. First, Tikhonov regularization set to $1e - 8$ is applied to improve the conditioning of covariance matrices and ensure numerical stability. Second, a recentering step aligns covariance matrices toward a common reference point (the identity) on the manifold in order to reduce inter-subject and inter-session variability. Finally, an equalization is performed to normalize the covariance structure. For datasets with more than 100 channels, a reduction of dimensions is performed via a PCA. The resulting dimension is 96 channels.

5.3 Library SPD Learn

The implementation of the proposed architecture is based on the SPD Learn library, a Pytorch-based framework dedicated to geometric deep learning on SPD matrices. It provides differentiable Riemannian operations, layers designed to operate on the SPD manifold, and implementations of several published neural network architectures [27].

In this study, the architecture adopted follows the structure of the SPDNet model. The network is composed of several layers, provided by the library, such as the BiMap layer, ReEig layer, and the LogEig layer. In the standard formulation, non-linearity is introduced through the ReEig layer, which performs eigenvalue rectification to ensure positive definiteness.

The tested activation functions are integrated in the same position as the ReEig layer within the SPDNet architecture, ensuring a consistent comparison between the non-linear transformations.

5.4 Model

Based on the SPDNet architecture implemented in the SPD Learn library, different activation functions are integrated and compared within a Riemannian neural network framework. This section describes the model architecture and the training configuration.

The overall architecture, presented in Figure 6, is composed of two main parts : a domain-dependant block and a domain-independent block. The input to the model consists of covariance matrices computed from EEG trials. The first block corresponds to a SPD block, as represented in Figure 7, composed of a sequence of BiMap layers followed by non-linear activation layers. The second block consists of a LogEig layer, followed by a linear classifier. The network outputs raw logits from the final linear layer. During training, probabilities are obtained implicitly through

the cross-entropy loss function. During the inference phase, the predicted class is obtained by applying an argmax over the output logits of the classifier.

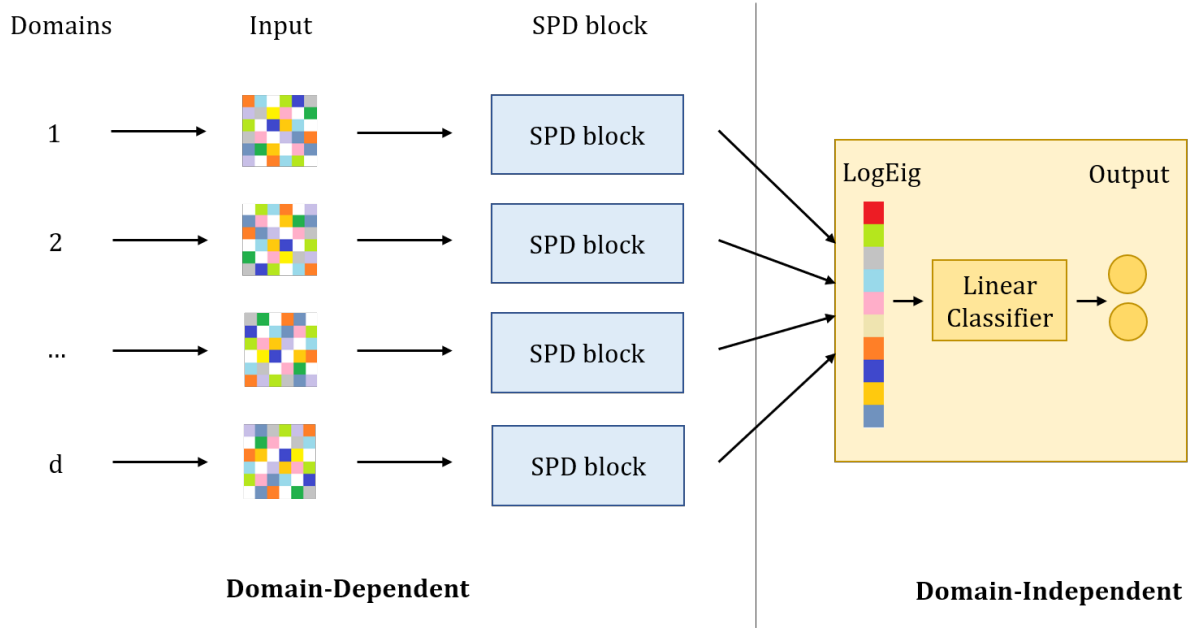


Figure 6: SPDNet model.

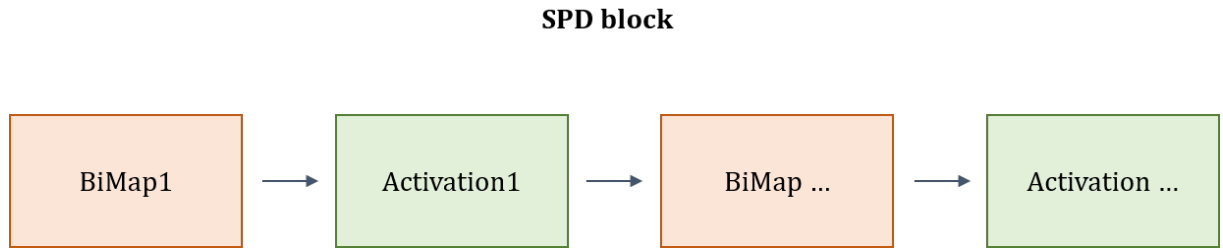


Figure 7: SPDNet block.

The number of BiMap and activation layers is adapted to the dimensions of input covariance matrices in order to control the network complexity. Each BiMap layer reduces the dimensionality by a factor of two. For dataset with originally more than 60 and 120 EEG channels, two and three BiMap and activation layers are employed.

Training is performed using the Riemannian ADAM optimizer from the library Geoopt [28] with a cross-entropy loss function. A batch size of 32, a learning rate of 0.005 and a maximum of 75 epochs were used. All hyperparameters are consistent across subjects and datasets to ensure a fair comparison between activation functions. The model is evaluated using a within-subject procedure.

5.5 Activation Layers

In this section, we detail the non-linear activation functions investigated in this study, such as the spectral-based activations and element-wise activations, defined in section 4.5. In the first category, three spectral transformations are presented, such as PowerEig, TanhEig and

SpAEig. In the second category, four activation functions are introduced. All the non-linearities investigated are illustrated in Figure 8.

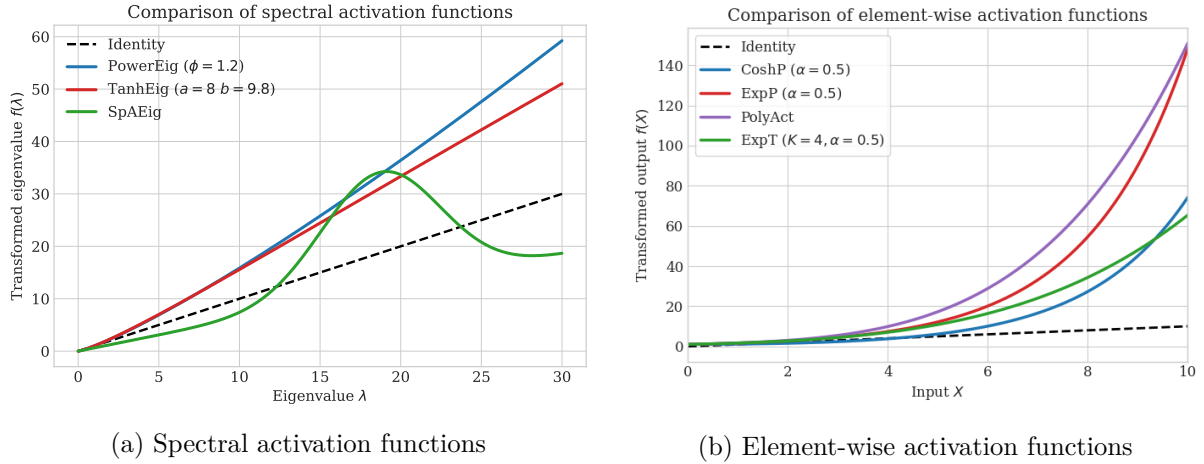


Figure 8: Representation of spectral and element-wise activation functions applied to SPD matrices.

For each activation function, both the forward and backward operations are detailed in the next subsections. In the equations of the powerEig, TanhEig, and SpAEig layers, the masking effect induced by the eigenvalue clamping operation is omitted for simplicity. Consequently, the corresponding gradients are presented without the clamp subgradient term.

5.5.1 PowerEig

The PowerEig activation function raises the eigenvalues of a SPD matrices to the power ω .

Forward

In the eigenspace, the PowerEig layer applies the following transformation to the eigenvalues :

$$\max(\epsilon, \lambda^\omega),$$

where ϵ is a lower threshold used to clamp small eigenvalues in order to preserve numerical stability and positive definiteness. In the study, ϵ is set to $1e - 4$. The parameter ω is learnt during training and controls spectrum geometry via a power transformation. It is constrained by a softplus function to ensure its positivity and is initialized at 1. For values of ω between 0 and 1, the transformation is compressive. Large eigenvalues are more decreased relatively to smaller. When $\omega = 1$, the transformation is equal to the identity and the spectrum remains unchanged. Values of ω larger than 1 enhance dominant eigenvalues. The influence of the parameter ω on the eigenvalue spectrum is illustrated in Figure 9.

Backward

The computation of the gradient of the loss with respect to the input matrix, $\frac{\partial L}{\partial X}$, relies on the "modeig_backward" function implemented in SPD Learn. This function requires the derivative

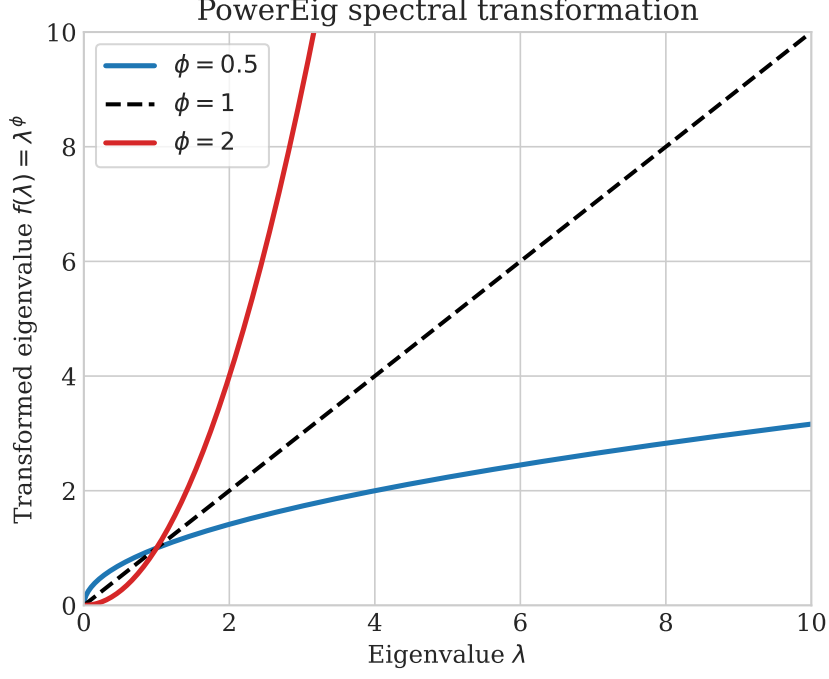


Figure 9: Effect of the parameter ϕ in the PowerEig layer.

of the transformation applied to the eigenvalues :

$$\frac{\partial \lambda^\omega}{\partial \lambda} = \omega \lambda^{\omega-1}.$$

Furthermore, in order to make the exponent ω learnable, the gradient of the loss with respect to ω is computed using the chain rule :

$$\frac{\partial L}{\partial \omega} = \sum_i \frac{\partial L}{\partial \lambda_i^\omega} \times \frac{\partial \lambda_i^\omega}{\partial \omega}.$$

The term

$$\frac{\partial L}{\partial \lambda_i^\omega} = [U^T \frac{\partial L}{\partial Y} U]_{ii},$$

corresponds to the projected gradient, where $\frac{\partial L}{\partial Y}$ is returned by the framework, while

$$\frac{\partial \lambda_i^\omega}{\partial \omega} = \lambda_i^\omega \log(\lambda_i).$$

Therefore,

$$\frac{\partial L}{\partial \omega} = \sum_i [U^T \frac{\partial L}{\partial Y} U]_{ii} \lambda_i^\omega \log(\lambda_i).$$

5.5.2 TanhEig

The TanhEig layer applies the hyperbolic tangent transformation to the eigenvalues of SPD matrices.

Forward

In the eigenspace, the TanhEig layer applies the following transformation to the eigenvalues:

$$\max\left(\epsilon, \exp\left(b \tanh\left(\frac{\log(\lambda)}{a}\right)\right)\right),$$

where ϵ is a lower threshold used to clamp small eigenvalues. The parameters a and b are learnt during training. Parameter a controls the horizontal scaling of the non-linearity. Small values of a cause the hyperbolic tangent function to saturate rapidly toward ± 1 , leading to a sharper transformation. Larger values of a produce a smoother and more gradual transition. Parameter b controls the amplitude of the transformation. Small values of b compress eigenvalues towards 1 whereas larger values of b increases the spectral range. As shown in Figure 10, the parameter a controls the saturation rate while b controls the activation amplitude. Both parameters are constrained by a softplus function to ensure their positivity, and are initialized to $a = 9$ and $b = 9.2103$, values for which the transformation $\exp(b \tanh(\frac{\log(\lambda)}{a}))$ behaves approximately as the identity.

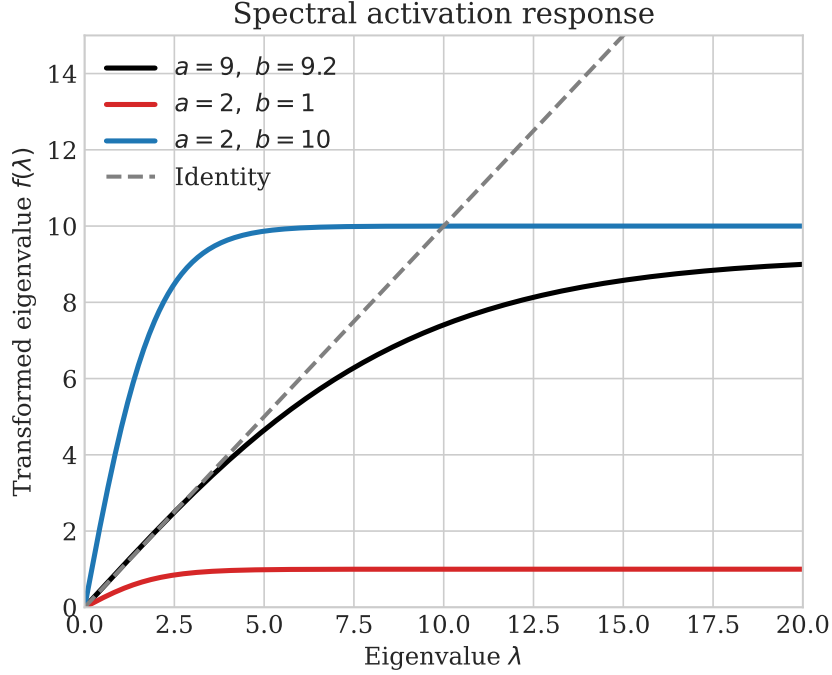


Figure 10: Simplified visualization of the TanhEig activation function illustrating the influence of the parameters a and b .

Backward

As explained in the previous section, the derivative of the function with respect to the eigenvalues is computed for the gradient $\frac{\partial L}{\partial X}$:

$$\frac{\partial f(\lambda)}{\partial \lambda} = \frac{b}{\lambda a} \frac{1}{\cosh^2\left(\frac{\log(\lambda)}{a}\right)} f(\lambda),$$

with $f(\lambda) = \exp(b \tanh(\frac{\log(\lambda)}{a}))$.

Furthermore, gradients of the loss with respect to the parameters are computed. Detailed calculations are available in Appendix A. The gradient of the loss with respect to a is defined by

$$\frac{\partial L}{\partial a} = \sum_i [U^T \frac{\partial L}{\partial Y} U]_{ii} \left(-b \frac{\log(\lambda_i)}{a^2} \frac{1}{\cosh^2(\frac{\log(\lambda_i)}{a})} f(\lambda_i) \right).$$

The gradient of the loss with respect to b is given by

$$\frac{\partial L}{\partial b} = \sum_i [U^T \frac{\partial L}{\partial Y} U]_{ii} \left(\tanh(\frac{\log(\lambda_i)}{a}) f(\lambda_i) \right).$$

5.5.3 SpAEig

The main idea behind the SpAEig layer is to learn a non-linear modulation of the eigenvalues of SPD matrices in the form of a gaussian envelope to be applied multiplicatively to the eigenvalue spectrum. Intuitively, if the discriminative information is concentrated around a particular region of the spectrum, the Gaussian increases the contribution of the corresponding eigenvalues while progressively attenuating the others. For example, if the informative signal lies around the third eigenvalue, the Gaussian envelope will peak near this region and decrease away from it. The spectral attention mechanism acts on the ordered eigenvalues via a learnable parametric Gaussian envelope.

When considering narrow-band covariance matrices, we assume that the information associated with a given frequency band concentrates on a single region of the eigenvalue spectrum. In this setting, we assume that a single Gaussian envelope is sufficient to model the spectral attention. If instead we consider large-band covariance matrices (8-35 Hz), the informative signal may spread across several regions of the spectrum and a multi-envelope with K Gaussian components will be needed. We consider the narrow-band case for the following forward and backward operations.

Forward

In the eigenspace, the SpAEig layer applies a modulation of the eigenvalues defined as

$$\max(\epsilon, \lambda_i g_{\text{norm}}),$$

where g_{norm} is a normalized Gaussian envelope defined as

$$g_{\text{norm}} = \frac{g_i}{\bar{g}},$$

with g_i a Gaussian envelope defined over the n eigenvalue index i , such as

$$g_i = 1 + A \exp\left(-\frac{(i - \mu)^2}{2 \sigma^2}\right),$$

and the normalization factor :

$$\bar{g} = \frac{1}{n} \sum_{j=1}^n g_j.$$

ϵ is a lower threshold used to clamp small eigenvalues in order to preserve the numerical stability and positive definiteness. In our experiment, ϵ is set to $1e - 4$. The parameters μ , σ and A

are learnt during training and respectively control the location, width, and the amplitude of the Gaussian envelope. Figure 11 illustrates how these parameters modulates the spectral attention mechanism. All parameters are constrained to ensure that $\mu \in (0, n)$ with n being the number of eigenvalues, $\sigma > 0$, and $A \geq 0$:

$$\mu = n \operatorname{sigmoid}(\theta_\mu),$$

$$\sigma = \operatorname{softplus}(\theta_\sigma) + \epsilon,$$

$$A = \operatorname{softplus}(\theta_A),$$

with θ_μ , θ_σ , and θ_A unconstrained variables.

These parameters are initialized at $\mu = n - 2$, $\sigma = 3$, and $A = 1$.

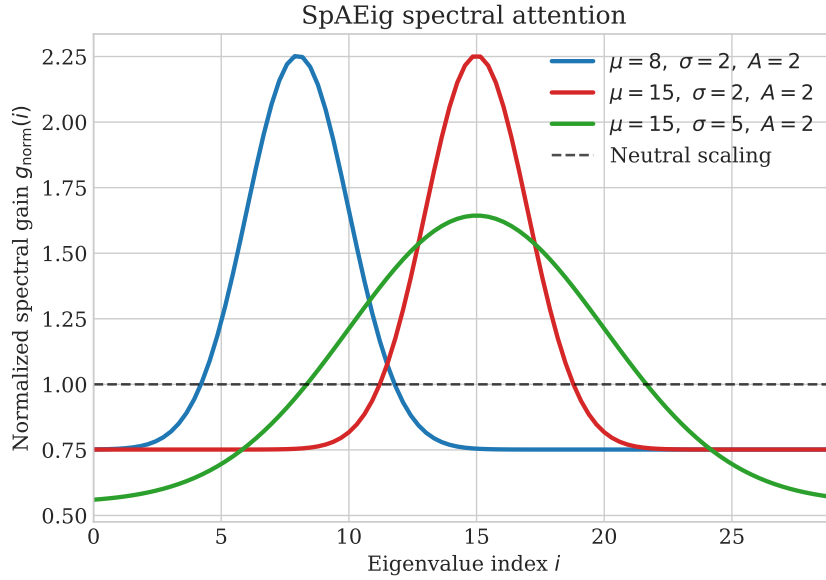


Figure 11: Spectral attention envelopes for different values of μ , σ , and A .

Backward

The derivative of the function with respect to the eigenvalues is computed for the calculation of the gradient $\frac{\partial L}{\partial X}$:

$$\frac{\partial}{\partial \lambda_i} \lambda_i g_{\text{norm}} = g_{\text{norm}}.$$

Furthermore, gradients of the loss with respect to the parameters are computed, and detailed calculations are available in Appendix B.

The gradient of the loss with respect to μ is defined as :

$$\frac{\partial L}{\partial \mu} = \sum_i \left[U^T \frac{\partial L}{\partial Y} U \right]_{ii} \lambda_i \frac{A}{\sigma^2} \left[\frac{(i - \mu) E_i}{\bar{g}} - \frac{g_i}{\bar{g}^2} \bar{E}_\mu \right],$$

where

$$E_i = \exp\left(-\frac{(i - \mu)^2}{2\sigma^2}\right),$$

and

$$\bar{E}_\mu = \frac{1}{n} \sum_{j=1}^n (j - \mu) E_j.$$

The gradient of the loss with respect to σ is given by

$$\frac{\partial L}{\partial \sigma} = \sum_i \left[U^T \frac{\partial L}{\partial Y} U \right]_{ii} \lambda_i \frac{A}{\sigma^3} \left[\frac{(i - \mu)^2 E_i}{\bar{g}} - \frac{g_i}{\bar{g}^2} \bar{E}_\sigma \right],$$

where

$$\bar{E}_\sigma = \frac{1}{n} \sum_{j=1}^n (j - \mu)^2 E_j.$$

Finally, the gradient of the loss with respect to A is defined as :

$$\frac{\partial L}{\partial A} = \sum_i \left[U^T \frac{\partial L}{\partial Y} U \right]_{ii} \lambda_i \left(\frac{E_i}{\bar{g}} - \frac{g_i}{\bar{g}^2} \bar{E} \right),$$

where

$$\bar{E} = \frac{1}{n} \sum_{j=1}^n E_j.$$

The following non-linearities introduced are element-wise activations, applying the transformation directly to the entries of SPD matrices. These functions admit Maclaurin expansions with non-negative coefficients, a sufficient condition for preserving positive definiteness under entry-wise application. We define the operator $\circ$ as the entry-wise operation.

5.5.4 Parametric Hyperbolic Cosine - CoshP

Forward

The parametric hyperbolic cosine CoshP applies the following function to the SPD matrix X :

$$Y = \cosh \circ (\alpha X),$$

where α is a learnable parameter constrained with a softplus function and initialized to 0.5.

Backward

The gradient of the loss with respect to the input is computed using the chain rule

$$\frac{\partial L}{\partial X_{ij}} = \frac{\partial L}{\partial Y_{ij}} \times \frac{\partial Y_{ij}}{\partial X_{ij}}.$$

Therefore, $\frac{\partial L}{\partial X}$ is expressed as

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \odot \sinh \circ (\alpha X) \alpha,$$

where $\frac{\partial L}{\partial Y}$ is returned by the framework.

The gradient with respect to the parameter α is given by

$$\frac{\partial L}{\partial \alpha} = \sum_{ij} \frac{\partial L}{\partial Y_{ij}} \sinh \circ (\alpha X_{ij}) X_{ij}.$$

5.5.5 Polynomial Activation - PolyAct

Forward

This approach consists in approximating a non-linear function with a polynomial defined by :

$$Y = \sum_{k=0}^K a_k X^{\circ k}.$$

The coefficients a_k are learnt during training and constrained using a soft-plus function to ensure non-negativity.

Backward

The gradient of the loss with respect to the input is defined by :

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \odot \left(\sum_{k=1}^K a_k k X^{\circ(k-1)} \right).$$

The gradient of the loss with respect to the coefficient a_k is defined by :

$$\frac{\partial L}{\partial a_k} = \sum_{ij} \frac{\partial L}{\partial Y_{ij}} X_{ij}^k.$$

5.5.6 Parametric Exponential Activation - ExpP

Forward

The function ExpP apply the following transformation to the input matrix :

$$Y = \exp \circ (\alpha X).$$

where α is a learnable parameter initialized to 0.5, and constrained by a soft-plus function.

Backward

Using the chain rule, the gradient $\frac{\partial L}{\partial X}$ is defined by :

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \odot \alpha \exp(\alpha X).$$

The gradient with respect to the parameter α is given by :

$$\frac{\partial L}{\partial \alpha} = \sum_{ij} \frac{\partial L}{\partial Y_{ij}} X_{ij} \exp \circ (\alpha X_{ij}).$$

5.5.7 Truncated Exponential - ExpT

This function corresponds to a truncated approximation of $e^{\alpha X}$.

Forward

In this approach, a single parameter is learnt for the full polynomial, defined by :

$$Y = \sum_{k=0}^K \alpha_k X^{\circ k},$$

with $a_k = \frac{\alpha^k}{k!}$. The parameter α is learnt during training, is constrained by a soft-plus function and initialized to 0.5.

Backward

Using the chain rule, the gradient $\frac{\partial L}{\partial X}$ is defined by

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \odot \left(\sum_{k=1}^K \frac{a^k}{k!} k X^{\circ(k-1)} \right).$$

The gradient with respect to the parameter α is given by

$$\frac{\partial L}{\partial \alpha} = \sum_{ij} \frac{\partial L}{\partial Y_{ij}} \left(\sum_{k=1}^K \frac{a^{k-1}}{(k-1)!} X_{ij}^k \right).$$

5.6 Statistical Tests

In order to assess the influence of some of the activation functions investigated on the classification performance, statistical tests are conducted. All architectures share exactly the same configuration, except for the activation layer. For each dataset, the experiments were repeated using five different random seeds to account for the variability introduced by the initialization of the weights. The models were trained and evaluated on the five folds of each dataset. For each pair (seed, fold), the activation functions were evaluated under the same experimental conditions. The metric used was the balanced-accuracy, calculated on the test set. This procedure provides for each activation 25 observations. As the performance for the different activation functions is calculated on the same (seed, fold) pairs, the observations are paired and constitute a repeated-measure design.

Initially, an Omnibus test was carried out to determine whether there was a significant difference between the means of all activation functions. This analysis was performed using the AnovaT-testRM function from the package PermutationTests.jl [29], which implements a permutation-based repeated-measures analysis of variance. The null hypothesis tested is

$$H_0 : \mu_{\text{ReEig}} = \mu_{\text{activation1}} = \dots = \mu_{\text{activationk}},$$

where μ represents the mean performance of each activation function and k the number of activation functions tested.

When the Omnibus test revealed a significant difference ($p < 0.05$), a Post-Hoc test was car-

ried out using the `studentMcTestRM` function from the package `PermutationTests.jl` [29], to determine which mean deviates. For each pair of activation functions, differences in balanced accuracy were calculated using the same observations (same fold and same seed). This test performs pairwise comparisons on matched data. A correction for multiple comparisons was applied to control the family-wise error rate and reduce the risk of false positives. Particular attention was paid to comparisons involving the ReEig layer, which was used as a reference in this study.

The proposed activation functions were therefore evaluated both in terms of classification performance and statistical significance with respect to the baseline ReEig activation.

6 Results

This section presents the experimental results obtained with the different activation functions integrated into the SPDNet architecture. First, results of the classification performance are presented, followed by the statistical analysis.

6.1 Classification performance

This subsection presents the classification performance obtained with the different activation functions. Results are reported as the mean balanced accuracy $\pm$ standard deviation (std) over the five folds of each dataset. Table 2 summarizes the classification performance obtained when the SPD block is composed of a single BiMap layer, followed by a single activation layer, as described in section 5.4.

Table 2: Mean balanced accuracy ($\% \pm$ std) of different activation functions across datasets.

Activation	BNCI2014	BNCI2015	Weibo2014	Zhou2016	Cho2017	Mean
ExpT	85.83 $\pm$ 00.91	81.27 $\pm$ 00.95	81.56 $\pm$ 01.84	90.13 $\pm$ 02.18	65.71 $\pm$ 02.27	80.90
ExpP	85.40 $\pm$ 01.02	80.88 $\pm$ 00.76	81.27 $\pm$ 02.01	88.21 $\pm$ 02.59	65.72 $\pm$ 01.90	80.30
CoshP	85.06 $\pm$ 01.18	81.01 $\pm$ 00.66	78.33 $\pm$ 01.51	89.12 $\pm$ 01.42	65.72 $\pm$ 01.79	79.85
Cosh	84.22 $\pm$ 00.86	80.68 $\pm$ 00.85	79.46 $\pm$ 01.55	88.32 $\pm$ 01.80	64.69 $\pm$ 02.11	79.47
ReEig	84.50 $\pm$ 00.15	79.43 $\pm$ 01.21	80.59 $\pm$ 01.57	89.41 $\pm$ 01.60	61.32 $\pm$ 02.12	79.05
TanhEig	84.32 $\pm$ 01.38	80.44 $\pm$ 00.60	78.95 $\pm$ 01.74	89.25 $\pm$ 00.79	59.69 $\pm$ 02.51	78.53
SpAEig	84.69 $\pm$ 01.59	80.44 $\pm$ 00.44	78.23 $\pm$ 03.12	90.32 $\pm$ 02.07	56.05 $\pm$ 00.82	77.95
PowerEig	82.92 $\pm$ 01.74	79.74 $\pm$ 00.78	78.36 $\pm$ 03.09	88.75 $\pm$ 01.67	58.63 $\pm$ 01.56	77.68
PolyAct	83.81 $\pm$ 01.23	73.86 $\pm$ 00.95	77.58 $\pm$ 02.58	86.04 $\pm$ 02.63	58.78 $\pm$ 03.19	76.01

Overall, several among the proposed activation functions achieve performances comparable to or higher than the baseline ReEig activation. ExpT obtains the highest average balanced accuracy accross the five datasets, followed by ExpP and CoshP. At the dataset level, ExpT achieves the highest performance on BNCI2014, BNCI2015, and Weibo2014, while SpAEig obtains the highest score on Zhou2016. For Cho2017, ExpP and CoshP achieve the highest balanced accuracy. During training, parameters of activation layers are monitored over the five folds of each dataset for each domain. For each activation layer, the minimum and maximum values at the end of the training phase are summarized in Table 3. Parameters for SpAEig and Polyact are not reported in the table because their format is not adapted.

Table 3: Parameters monitoring during training on different datasets.

Activation Layer	Parameter	BNCI2014	BNCI2015	Weibo2014	Zhou2016	Cho2017
ReEig	Clamped values	None	None	None	None	None
ExpT	$\alpha = 0.5$	[0.37 ; 0.60]	[0.35 ; 0.75]	[0.40 ; 0.50]	[0.37 ; 0.77]	[0.44 ; 0.50]
ExpP	$\alpha = 0.5$	[0.42 ; 0.63]	[0.45 ; 0.65]	[0.40 ; 0.50]	[0.40 ; 0.80]	[0.42 ; 0.50]
CoshP	$\alpha = 0.5$	[0.10 ; 0.85]	[0.30 ; 1.06]	[0.45 ; 0.71]	[0.23 ; 1.00]	[0.40 ; 0.67]
PowerEig	$\omega = 1.0$	[1.07 ; 1.15]	[0.80 ; 1.18]	[1.10 ; 1.34]	[1.02 ; 1.18]	[1.12 ; 1.23]
TanhEig	$a = 9.0$	[8.65 ; 8.80]	[8.60 ; 9.10]	[8.10 ; 8.12]	[8.65 ; 8.90]	[1.12 ; 1.23]
	$b = 9.21$	[9.36 ; 9.71]	[9.24 ; 9.60]	[9.35 ; 9.55]	[9.50 ; 9.90]	[1.12 ; 1.23]

Table 3 shows that the parameters of the proposed activation functions remains stable throughout training. We see that the range of values is consistent for both ExpP and ExpT, more so than CoshP. Across all datasets, no eigenvalues are clamped by the ReEig activation.

To further evaluate the most promising activation functions, additional evaluations were conducted on three supplementary datasets with large number of electrodes. The corresponding results are reported in Table 4.

Table 4: Mean balanced accuracy ($\% \pm \text{std}$) of different activation functions over supplementary datasets.

Activation	Lee2019	GrosseWentrup2009	Schirrmeister2017	Mean
ExpT	70.88 $\pm$ 01.55	80.85 $\pm$ 01.99	80.51 $\pm$ 00.95	77.21
CoshP	71.34 $\pm$ 01.49	75.92 $\pm$ 01.51	79.23 $\pm$ 00.78	75.49
Cosh	67.93 $\pm$ 03.13	77.34 $\pm$ 01.50	78.75 $\pm$ 01.37	74.67
ReEig	63.99 $\pm$ 04.15	79.71 $\pm$ 01.29	79.23 $\pm$ 01.36	74.31

Table 4 confirms the trend observed on the initial benchmark datasets. ExpT achieves the highest average performance across these datasets, outperforming both the baseline ReEig and the other evaluated activations. While CoshP slightly surpasses ExpT on Lee2019, ExpT obtains the best results on both GrosseWentrup2009 and Schirrmeister2017.

The influence of network depth was then investigated by adapting the number of BiMap and activations layers to the dimensionality of the covariance matrices. The main idea is to enforce linear combinations of non-linear transformations. For datasets with more than 60 channels, the SPD block is composed of two sequence of BiMap layer and activation layer. An additional BiMap-activation block is added for datasets with originally more than 120 channels. Results are reported in Table 5 and 6.

Table 5: Mean balanced accuracy ($\% \pm \text{std}$) of different activation functions over datasets with more than 60 channels.

Activation	Lee2019	Weibo2014	Cho2017	Mean
ExpT	73.57 $\pm$ 01.75	82.26 $\pm$ 02.06	69.13 $\pm$ 00.83	74.99
CoshP	71.83 $\pm$ 02.03	82.76 $\pm$ 01.39	67.83 $\pm$ 01.67	74.14
Cosh	70.49 $\pm$ 01.48	81.55 $\pm$ 01.52	66.67 $\pm$ 01.50	72.90
ReEig	68.38 $\pm$ 02.75	81.72 $\pm$ 01.83	64.34 $\pm$ 02.82	69.81

Table 6: Mean balanced accuracy ($\% \pm \text{std}$) of different activation functions over datasets with originally more than 120 channels.

Activation	GrosseWentrup2009	Schirrmeister2017	Mean
ReEig	84.88 $\pm$ 01.77	82.16 $\pm$ 01.13	83.52
ExpT	84.44 $\pm$ 01.54	82.70 $\pm$ 01.82	83.36

Increasing the depth of the SPD network generally improves performance for all activation

functions. For datasets with more than 60 channels (Table 5), ExpT remains the best-performing activation, followed by CoshP. Both activations outperform the ReEig baseline on Lee2019 and Cho2017. On Weibo2014, CoshP obtains the highest balanced accuracy.

For datasets with originally more than 120 channels (Table 6), CoshP and Cosh failed to converge under these conditions because of numerical instabilities arising from ill-conditioned covariances matrices and repeated eigenvalues. Among the methods that successfully converged, ExpT and ReEig achieve comparable performances.

6.2 Statistical analysis

This subsection presents the statistical analysis of the results. A permutation-based repeated-measures ANOVA was first applied to assess overall differences between activation functions (Omnibus test), followed by post-hoc pairwise comparisons when significance was detected. Statistical analysis is conducted on four different activations. The ReEig and Cosh activations represent the baseline whereas ExpT and CoshP represent the alternative functions to be tested.

6.2.1 Omnibus analysis

The omnibus ANOVA reveals a significant effect of the activation functions on classification performance for four of the five datasets (Table 7). Significant differences are observed for BNCI2014, BNCI2015, Weibo2014, and Cho2017, indicating that the choice of activation function has a measurable impact on performance. In contrast, no significant effect is detected for Zhou2016, suggesting that all activation functions achieve comparable performance on this dataset. Consequently, post-hoc analyses are only interpreted for datasets exhibiting significant omnibus effects.

Table 7: Omnibus permutation-based repeated-measures ANOVA results across datasets. The p-value indicates the significance of differences between at least two activation functions. Significance levels are indicated as: s (significant for p-value < 0.05), n.s. (not significant).

Dataset	Test Statistic	p-value	Significance
BNCI2014	1812.77	< 0.01	s
BNCI2015	1624.85	< 0.01	s
Weibo2014	1588.81	< 0.01	s
Zhou2016	1984.96	0.33	n.s.
Cho2017	962.12	< 0.01	s

6.2.2 Post-hoc analysis

To further investigate differences between activation functions, post-hoc tests were conducted on the datasets exhibiting significant omnibus effects. The detailed comparisons are reported in Table 8. Significant differences are observed in multiples cases.

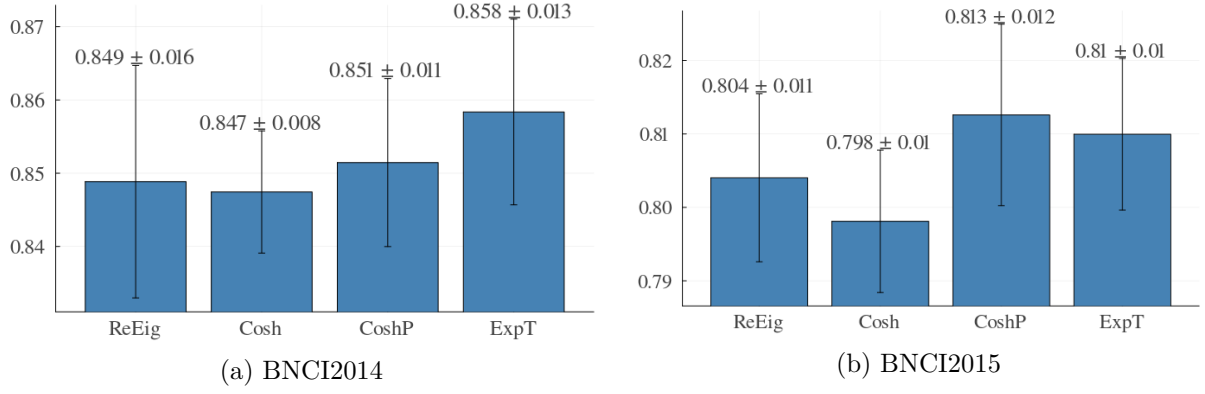


Figure 12: Balanced accuracy across BNCI2014 and BNCI2015 datasets.

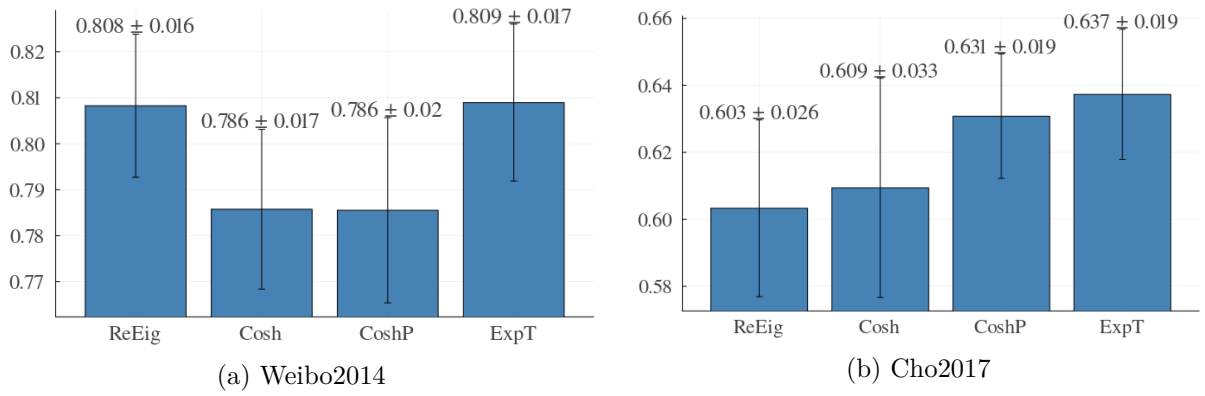


Figure 13: Balanced accuracy across Weibo2014 and Cho2017 datasets.

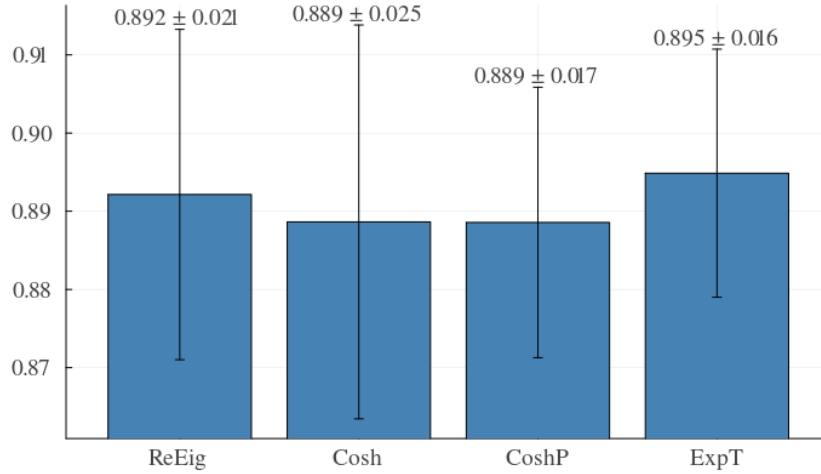


Figure 14: Balanced accuracy on Zhou2016.

Figures 12, 13, and 14 illustrate the distribution of balanced accuracies across folds for each activation function. For BNCI2014, BNCI2015, and Cho2017, ExpT and CoshP generally exhibit higher performance than the baseline activations. On Weibo2014, ExpT and ReEig show similar distributions, which is consistent with the absence of significant differences in the corresponding post-hoc comparison (Table 8). The overlap between distributions is substantially larger for Zhou2016, supporting the non-significant omnibus result reported in Table 7.

Table 8: Post-hoc pairwise comparisons between activation functions using permutation-based paired t-tests. The p-value indicates statistical significance after repeated-measures comparison. The signed mean difference corresponds to the mean difference in balanced accuracy (first method minus second method). Boldface indicates the activation function associated with the significantly higher balanced accuracy when the comparison is statistically significant ($p < 0.05$).

Database	Activation layers	P-value	Signed Mean Diff
BNCI2014	ReEig - Cosh	0.96	0.51
	ReEig - CoshP	0.84	-0.83
	ReEig - ExpT	0.05	-2.67
	Cosh - CoshP	0.16	-2.14
	Cosh - ExpT	<0.01	-4.27
	CoshP - ExpT	0.11	-2.34
BNCI2015	ReEig - Cosh	0.07	2.17
	ReEig - CoshP	0.03	-3.02
	ReEig - ExpT	0.02	-3.17
	Cosh - CoshP	<0.01	-6.69
	Cosh - ExpT	<0.01	-5.72
	CoshP - ExpT	0.25	1.52
Weibo2014	ReEig - Cosh	<0.01	5.51
	ReEig - CoshP	<0.01	5.33
	ReEig - ExpT	0.96	-0.23
	Cosh - CoshP	0.99	0.06
	Cosh - ExpT	<0.01	-7.10
	CoshP - ExpT	<0.01	-5.62
Cho2017	ReEig - Cosh	0.57	-0.94
	ReEig - CoshP	<0.01	-4.55
	ReEig - ExpT	<0.01	-5.41
	Cosh - CoshP	0.01	-3.32
	Cosh - ExpT	<0.01	-3.76
	CoshP - ExpT	0.30	-1.42

The post-hoc analysis provides further insight into the significant omnibus effect. For BNCI2014, ExpT significantly outperforms Cosh. On BNCI2015, both ExpT and CoshP significantly improved classification over the baseline ReEig activation. For Weibo2014, ExpT significantly outperforms both Cosh and CoshP, while no significant difference is observed between ExpT and ReEig. On Cho2017, both ExpT and CoshP significantly outperform ReEig and Cosh, while no significant difference is detected between ExpT and CoshP. Overall, the post-hoc comparison support the superiority of ExpT across multiple datasets.

7 Discussion

This study investigates the impact of non-linear activation functions within a Riemannian neural network (SPDNet) for EEG-base Motor Imagery classification. The objective was to evaluate whether replacing the baseline ReEig activation by new spectral activation functions or element-wise activation functions could improve classification performance. In this section, we summarize the results and present the interpretation behind. Further, we introduce de the limitations and the perspective of this study.

7.1 Summary of results

Overall, the experimental results show that the choice of the activation function has an influence on the classification performance across multiple datasets. Among all tested methods, ExpT consistently achieves the highest average performance or near-highest performance, on the main datasets and also on additional datasets. It is followed by the methods CoshP and ExpP, which show competitive results compared to the ReEig baseline method. In contrast, some activation functions such as PowerEig and TanhEig generally underperform. The ReEig activation remains a strong baseline, especially in deeper architectures where numerical instability becomes critical. Furthermore, adapting the depth of the network based on the number of EEG channels of datasets allow the classification performance to be increased in some cases. For Lee2019 and Cho2017, the gain is around three percent compared to the standard SPDNet with one SPD block (illustrated in 5.4). The statistical analysis proves that these performance differences are not due to randomness. Significant effects of activation functions are observed for multiple datasets. Post-hoc comparisons show that ExpT outperforms other alternatives. However, results show that no single activation achieves the highest average classification performance across all datasets.

7.2 Interpretation

The initial research question aimed at determining whether alternatives non-linear activation functions could improve classification performance in EEG-based BCIs over and above the standard ReEig activation. To answer this question we have evaluated a number of activation functions, some of which have been here proposed for the first time, on a very large amount of publicly available BCI databases. The results we have obtained suggest that the choice of the activation has a significant impact on the performance of a SPDnet. At the same time, the gain is not uniform and depends on the dataset characteristics such as the signal-to-noise ratio and number of EEG channels. The ExpT function we have designed appear to offer consistent superior performance. Moreover, monitoring the values clamped by the ReEig activation layer has shown that no non-linearity is introduced into the model. The resulting architecture can therefore be viewed as a sequence of learned BiMap projections followed by a classifier. Furthermore, we have investigated the effect on the classification performance of the number of SPD blocks composing the SPDnet. This makes sense when the number of electrodes of the data is large enough (>32). It appears clearly that increasing the number of blocks is beneficial, however,

deeper architectures might amplify numerical instability with some activation functions. This highlights a trade-off between expressivity and stability in SPD neural networks.

7.3 Limitations

Our study presents with several limitations. First of all, we have evaluated non-linear activation functions only for the SPDNet structure. Therefore, the results we have obtained cannot be generalized to other Riemannian network models. Second, our evaluations has exclusively concerned motor imagery data. Thus, the results cannot be generalized beyond the motor-imagery BCI paradigm.

7.4 Perspectives

Several perspectives could be considered for future work. A first direction would concern the instability of element-wise activation functions for deeper neural network. In this study, we have set the Tikhonov regularization to $1e-8$ and reduced the dimension of the databases exceeding 100 electrodes via a PCA. We have noticed numerical instabilities for these databases when no dimensionality reduction is applied. These instabilities may be evident or subtle. In the latter case, they may remain unnoticed, yet they could affect the performance. Thus, the appropriate regularization of the input covariance matrices is an import point that should not be neglected. The development of a standard regularization strategy is an interesting avenue for future research. In order to avoid instabilities with the element-wise activation functions such as ExpT and CoshP, one could clamp the values of the output in a reasonable range, for example, (0.0005-2000). The lower bound is typical in the ReEig layer, while the upper-bound corresponds to an amplitude of about 45 microVolts for the EEG (square root of 2000), which in humans is never surpassed for the sensorimotor rhythms exploited for motor imagery decoding. Another direction could be the extension of spectral attention mechanism, such as SpAEig, to a multiple Gaussian which could better capture the discriminative information in wide-bands EEG.

8 Conclusion

This work investigated through a comparative analysis and statistical analysis whether alternative activation functions can improve the performance of Riemannian neural networks on motor imagery classification. Spectral functions and element-wise functions were introduced to replace the ReEig standard activation. Experiments show that element-wise functions, particularly ExpT, achieves the highest average performance on multiple datasets, while CoshP and ExpP show competitive results. However, performance gains are not uniform across datasets, and classical ReEig remains a strong baseline in deep configuration with high-dimensional datasets. Therefore, the choice of the activation layer plays a significant role in SPD neural networks. Nevertheless, numerical instabilities were observed for some element-wise activations in deeper architecture. Future work could focus on improving numerical stability.

References

- [1] Joseph N. Mak and Jonathan R. Wolpaw. “Clinical Applications of Brain-Computer Interfaces: Current State and Future Prospects”. In: *IEEE Reviews in Biomedical Engineering* 2 (2009), pp. 187–199.
- [2] Marc Jeannerod. “Neural simulation of action: A unifying mechanism for motor cognition”. In: *NeuroImage* 14 (2001).
- [3] Santiago Galdo-Alvarez et al. “Functional equivalence of imagined vs. real performance of an inhibitory task: An EEG/ERP study”. In: *Frontiers in Human Neuroscience* 10 (2016).
- [4] Reinmar J. Kobler et al. “SPD-domain-specific batch normalization to crack interpretable unsupervised domain adaptation in EEG”. In: *IEEE Transactions on Biomedical Engineering* 10 (2020).
- [5] Marco Congedo, Alexandre Barachant, and Rajendra Bhatia. “Riemannian geometry for EEG-based brain-computer interfaces: a primer and a review”. In: *Brain-Computer Interfaces* 4.3 (2017), pp. 155–174.
- [6] Ce Ju et al. *SPD Matrix Learning for Neuroimaging Analysis: Perspectives, Methods, and Challenges*. 2026. arXiv: 2504.18882 [cs.LG]. URL: <https://arxiv.org/abs/2504.18882>.
- [7] GIPSA-lab. *Le Laboratoire*. Accessed: 2026-05-20. 2025. URL: <https://www.gipsa-lab.grenoble-inp.fr/le-laboratoire>.
- [8] GIPSA-lab. *Pôle Sciences des Données*. Accessed: 2026-05-20. 2025. URL: <https://www.gipsa-lab.grenoble-inp.fr/pole/pole-sciences-des-donnees-psd>.
- [9] GIPSA-lab. *ViBS Team – Vision and Brain Signal Processing*. Accessed: 2026-05-20. 2025. URL: <https://www.gipsa-lab.grenoble-inp.fr/equipe/vibs>.
- [10] Fabien Lotte et al. “A review of classification algorithms for EEG-based brain-computer interfaces”. In: *Journal of Neural Engineering* 4 (2007).
- [11] Sylvain Chevallier, Igor Carrara, Bruno Aristimunha, et al. *The largest EEG-based BCI reproducibility study for open science: the MOABB benchmark*. 2024. arXiv: 2404.15319 [eess.SP]. URL: <https://arxiv.org/abs/2404.15319>.
- [12] Fabien Lotte et al. “A review of classification algorithms for EEG-based brain-computer interfaces: A 10 year update”. In: *Journal of Neural Engineering* (2018).
- [13] Zhiwu Huang and Luc Van Gool. “A Riemannian Network for SPD Matrix Learning”. In: *AAAI Conference on Artificial Intelligence*. 2017.
- [14] Lucas Heck dos Santos et al. “Non-Linear Activation Functions for Deep Riemannian Neural Networks”. In: *Proceedings of the 34th European Symposium on Artificial Neural Networks (ESANN)*. HAL: hal-05616889. Bruges, Belgium, Apr. 2026, pp. 535–540. DOI: 10.14428/esann/2026.ES2026-312.
- [15] Alexandre Barachant et al. “Multiple Brain-Computer Interface Classification by Riemannian Geometry”. In: *IEEE Transactions on Biomedical Engineering* 59.4 (2012), pp. 920–928.

- [16] G. Pfurtscheller and F. H. Lopes da Silva. “Event-related EEG/MEG synchronization and desynchronization: basic principles”. In: *Clinical Neurophysiology* 110.11 (1999), pp. 1842–1857.
- [17] Herbert Ramoser, Johannes Müller-Gerking, and Gert Pfurtscheller. “Optimal Spatial Filtering of Single Trial EEG During Imagined Hand Movement”. In: *IEEE Transactions on Rehabilitation Engineering* 8.4 (2000), pp. 441–446.
- [18] Alexandre Barachant et al. “Common Spatial Pattern revisited by Riemannian geometry”. In: *IEEE International Workshop on Multimedia Signal Processing*. 2010, pp. 472–477.
- [19] Alexandre Barachant et al. “Riemannian geometry applied to BCI classification”. In: *Latent Variable Analysis and Signal Separation (LVA/ICA)*. 2010, pp. 629–636.
- [20] Alexandre Barachant et al. “Classification of covariance matrices using a Riemannian-based kernel for BCI applications”. In: *Neurocomputing* 122 (2013), pp. 172–178.
- [21] Gatien Darley and Stéphane Bonnet. “A Unified Framework for Matrix Backpropagation”. In: *IEEE Transactions on Neural Networks and Learning Systems* 37.1 (2026).
- [22] Silvère Bonnabel. “Stochastic Gradient Descent on Riemannian Manifolds”. In: *IEEE Transactions on Automatic Control* 58.9 (2013), pp. 2217–2229. DOI: 10.1109/TAC.2013.2254619.
- [23] Mario Lezcano-Casado. *Trivializations for Gradient-Based Optimization on Manifolds*. 2019. arXiv: 1909.09501 [cs.LG]. URL: <https://arxiv.org/abs/1909.09501>.
- [24] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.
- [25] Fahim Doumi, Marco Congedo, and Antonio Esposito. *FII BCI Corpus Overview*. Documentation consulted on 2026-05-30. 2025. URL: <https://marco-congedo.github.io/Eegle.jl/dev/documents/FII%20BCI%20Corpus%20Overview>.
- [26] Marco Congedo. *Pseudo Affine-Invariant Riemannian Metrics for Efficient Brain-Computer Interfaces*. Tech. rep. Version v1. GIPSA-lab, University Grenoble Alpes, CNRS, Grenoble-INP, May 2026. URL: <https://hal.science/hal-05616892>.
- [27] Bruno Aristimunha et al. *SPD Learn: A Geometric Deep Learning Python Library for Neural Decoding Through Trivialization*. 2026. arXiv: 2602.22895 [q-bio.NC]. URL: <https://arxiv.org/abs/2602.22895>.
- [28] Max Kochurov, Rasul Karimov, and Serge Kozlukov. *Geoopt: Riemannian Optimization in PyTorch*. 2020. arXiv: 2005.02819 [cs.CG]. URL: <https://arxiv.org/abs/2005.02819>.
- [29] Marco Congedo. *PermutationTests.jl: Permutation-Based Statistical Tests in Julia*. Accessed: 2026-05-20. 2025. URL: <https://marco-congedo.github.io/PermutationTests.jl/stable/>.

Appendices

A Gradient derivation for the TanhEig layer

The non-linear transformation applied to the eigenvalues is

$$f(\lambda) = \exp(b \tanh(\frac{\log(\lambda)}{a})).$$

For the backward operation, in order to make the parameters a and b learnable, we compute the gradients of the loss with respect to the parameters.

Gradient of the loss with respect to a

The chain rule gives

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial f(\lambda)} \times \frac{\partial f(\lambda)}{\partial a}.$$

The gradient with respect to the transformed eigenvalues is obtained from the projected gradient

$$\frac{\partial L}{\partial f(\lambda_i)} = [U^T \frac{\partial L}{\partial Y} U]_{ii},$$

while

$$\frac{\partial f(\lambda)}{\partial a} = -b \frac{\log(\lambda)}{a^2} \frac{1}{\cosh^2(\frac{\log(\lambda)}{a})} f(\lambda).$$

Therefore,

$$\frac{\partial L}{\partial a} = \sum_i [U^T \frac{\partial L}{\partial Y} U]_{ii} - b \frac{\log(\lambda_i)}{a^2} \frac{1}{\cosh^2(\frac{\log(\lambda_i)}{a})} f(\lambda_i).$$

Gradient of the loss with respect to b

Similarly,

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial f(\lambda)} \times \frac{\partial f(\lambda)}{\partial b},$$

with

$$\frac{\partial f(\lambda)}{\partial b} = \tanh(\frac{\log(\lambda)}{a}) f(\lambda).$$

Hence,

$$\frac{\partial L}{\partial b} = \sum_i [U^T \frac{\partial L}{\partial Y} U]_{ii} \tanh(\frac{\log(\lambda_i)}{a}) f(\lambda_i).$$

B Gradient derivation for the SpAEig layer

In the eigenspace, the SpAEig layer applies a modulation of the eigenvalues defined as

$$\max(\epsilon, \lambda_i g_{\text{norm}}),$$

where g_{norm} is a normalized Gaussian envelope defined as

$$g_{\text{norm}} = \frac{g_i}{\bar{g}},$$

with g_i a Gaussian envelope defined over the n eigenvalue index i , such as,

$$g_i = 1 + A \exp\left(-\frac{(i - \mu)^2}{2 \sigma^2}\right),$$

and the normalization factor :

$$\bar{g} = \frac{1}{n} \sum_{j=1}^n g_j.$$

Gradients of the loss with respect to the parameters μ , σ , and A are computed for the backward operation.

Gradient of the loss with respect to μ

The chain rule gives :

$$\frac{\partial L}{\partial \mu} = \frac{\partial L}{\partial \tilde{\lambda}_i} \times \frac{\partial \tilde{\lambda}_i}{\partial \mu},$$

with $\tilde{\lambda}_i = \lambda_i g_{\text{norm}}$.

The gradient with respect to the transformed eigenvalues is obtained from the projected gradient

$$\frac{\partial L}{\partial \tilde{\lambda}_i} = [U^T \frac{\partial L}{\partial Y} U]_{ii},$$

while

$$\frac{\partial \tilde{\lambda}_i}{\partial \mu} = \lambda_i \frac{A}{\sigma^2} \left[\frac{(i - \mu) E_i}{\bar{g}} - \frac{g_i}{\bar{g}^2} \bar{E}_\mu \right],$$

with

$$E_i = \exp\left(-\frac{(i - \mu)^2}{2 \sigma^2}\right),$$

and

$$\bar{E}_\mu = \frac{1}{n} \sum_{j=1}^n (j - \mu) E_j.$$

Therefore,

$$\frac{\partial L}{\partial \mu} = \sum_i \left[U^T \frac{\partial L}{\partial Y} U \right]_{ii} \lambda_i \frac{A}{\sigma^2} \left[\frac{(i - \mu) E_i}{\bar{g}} - \frac{g_i}{\bar{g}^2} \bar{E}_\mu \right].$$

Gradient of the loss with respect to σ

Similarly,

$$\frac{\partial L}{\partial \sigma} = \frac{\partial L}{\partial \tilde{\lambda}_i} \times \frac{\partial \tilde{\lambda}_i}{\partial \sigma}.$$

with

$$\frac{\partial \tilde{\lambda}_i}{\partial \sigma} = \lambda_i \frac{A}{\sigma^3} \left[\frac{(i - \mu)^2 E_i}{\bar{g}} - \frac{g_i}{\bar{g}^2} \bar{E}_\sigma \right],$$

where

$$E_i = \exp\left(-\frac{(i - \mu)^2}{2\sigma^2}\right),$$

and

$$\bar{E}_\sigma = \frac{1}{n} \sum_{j=1}^n (j - \mu)^2 E_j.$$

Therefore,

$$\frac{\partial L}{\partial \sigma} = \sum_i \left[U^T \frac{\partial L}{\partial Y} U \right]_{ii} \lambda_i \frac{A}{\sigma^3} \left[\frac{(i - \mu)^2 E_i}{\bar{g}} - \frac{g_i}{\bar{g}^2} \bar{E}_\sigma \right].$$

Gradient of the loss with respect to A

Similarly,

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial \tilde{\lambda}_i} \times \frac{\partial \tilde{\lambda}_i}{\partial A}.$$

with

$$\frac{\partial \tilde{\lambda}_i}{\partial A} = \lambda_i \left(\frac{E_i}{\bar{g}} - \frac{g_i}{\bar{g}^2} \bar{E} \right),$$

where

$$E_i = \exp\left(-\frac{(i - \mu)^2}{2\sigma^2}\right)$$

and

$$\bar{E} = \frac{1}{n} \sum_{j=1}^n E_j.$$

Hence,

$$\frac{\partial L}{\partial A} = \sum_i \left[U^T \frac{\partial L}{\partial Y} U \right]_{ii} \lambda_i \left(\frac{E_i}{\bar{g}} - \frac{g_i}{\bar{g}^2} \bar{E} \right).$$

Student: ANDRIEU Emma

Master: SIGMA

Academic year: 2025/2026

Internship title: Universal Machine-Learning Models : applications to Brain-Computer Interfaces

Dates: From 01/02/2026 to 30/06/2026

Laboratory supervisor: Marco CONGEDO, marco.congedo@gipsa-lab.grenoble-inp.fr

Phelma Tutor: Ronald PHLYPO, ronald.phlypo@grenoble-inp.fr

Internship description:

The student will acquire familiarity with EEG data and, particularly, the BCI data to be used for training and testing the UMLMs. The student will study existing Riemannian deep neuronal networks and propose architectures that may serve the objectives of the project. Finally, the student will evaluate the performance of competing architectures, including published one and architectures currently under development at GIPSA-lab. The evaluations will be carried out on graphic card clusters available in Grenoble (GRICAD).

Resources and Support staff:

The work was performed using standard development tools, including Python and the Pytorch framework, as well as the SPD Learn library for Riemannian neural networks. Code development was conducted using Visual Studio Code.

In terms of supervision and collaboration, this internship was supervised by Marco Congedo. In addition, I worked with Isabelle Costa Maia (PhD student), who provided regular technical guidance and support in understanding of Riemannian methods.